15 動態規劃

Part IV Advanced Design and Analysis Techniques

本章定位

動態規劃把大量重複的遞迴呼叫合併為狀態表。難點是證明狀態沒有遺失未來決策，並從最佳值重建具體解。本章以切割、矩陣括號、共同子序列及搜尋樹建立這套方法。

前置知識

遞迴、最佳化與子問題重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立 rod cutting 與 LCS 的最佳子結構。
- 用維度與計算次序設計 DP 表。
- 推導 matrix-chain 與 optimal BST 的區間 DP。
- 由 choice 表完成重建，並判斷 DP 條件在陌生變形中是否仍成立。
- 15.1 Rod cutting
- 15.2 Matrix-chain multiplication
- 15.3 Elements of dynamic programming
- 15.4 Longest common subsequence
- 15.5 Optimal binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.1 Rod cutting

給定非負整數長度 n，p_i 是完整長度 i 木料的售價。每段須為正整數長度，允許不切割，整根必須出售，切割本身無成本；目標是最大總收益。

對長度 n，遞迴的第一個決策是「先切下第一段 i，再最佳處理剩餘 n-i」。令 `r_0=0`，

`r_n=max_{1<=i<=n}(p_i+r_{n-i})`。

必要性。 取任一切法的最佳收益 `r_n`，其第一個 piece 長 i。若剩餘 n-i 不是最佳，換成收益較高的 `r_{n-i}`，原切法仍合法且總收益更大，矛盾。所以某個最佳解必由 i 與最佳剩餘解組成。

充分性。 對每組候選 i，`p_i+r_{n-i}` 都是合法切法，因此不會大於真正最佳值。兩向不等式相等，recurrence 正確。

不快取的遞迴共有 $T(0)=1$、$T(n)=1+\sum_{j=0}^{n-1}T(j)=2^n$ 次呼叫；memo 或底層表把每個大小只算一次，轉移數 $\sum_{j=1}^{n} j=\Theta(n^2)$。這裡 T 是呼叫數，不是價格或收益。

自頂向下快取與由小到大填表

以下以 1-based 價格 p 為輸入，先算長度較小的剩餘問題。即使價格為負，長度 n 仍須完整出售，故每個非空狀態從 $-\infty$ 開始，不能默許丟棄木料。

```
r[0] = 0
for length = 1..n:
    r[length] = -∞
    for first = 1..length:
        value = p[first] + r[length-first]
        if value > r[length]:
            r[length] = value
            choice[length] = first
pieces = []
while n > 0:
    append choice[n] to pieces
    n = n-choice[n]
```

迴圈不變量是進入 length 時，所有 0..length−1 的 r 都等於該長度最優收益。內圈掃完時，每個合法第一段都被比較，轉移定理使 r[length] 正確。回溯每次至少減一，最多輸出 n 段。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Rod Cutting 的完整表與切割重建

取 `p=[1,5,8,9]`，`first[n]` 記錄取得最大值的第一段長度：

n	候選 p_i+r[n-i]	r[n]	first[n]
0	空	0	0
1	1	1	1
2	5, 1+1=2	5	2
3	8, 1+5=6, 5+1=6	8	3
4	9, 1+8=9, 5+5=10, 8+1=9	10	2

回溯由 n=4 開始：`first[4]=2`，收益加上 `p_2=5`，剩長 2；`first[2]=2`，再加 5，剩 0。故最佳切法 `2+2`，總收益 10。表已算完才回溯，不能邊猜切點邊改動已定的最優值。

若每次切割收固定 c，可在尚有剩餘長度時對候選扣 c，單一長度狀態仍可用；若最多 k 段，則必須加入段數維度。是否須增加狀態，要看額外限制是否影響未來可行決策，不能只因規格改變就一概增加維度。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.2 Matrix-chain multiplication

A_i 的維度為 $p_{i-1} \times p_i$。若 A_i..A_j 的最後一次乘法在 k 與 k+1 間切開，成本為

`m[i,j]=m[i,k]+m[k+1,j]+p_{i-1}p_k p_j`。

對每個 k<j 取最小，且 `m[i,i]=0`。

最優解的左右子鏈必各自最優：若左子鏈有更便宜括號，換進去後最後乘法的成本不變，整條鏈成本反而下降，與最優性矛盾；右側同理。所有合法完全括號都有唯一的「最後一次切點」，所以枚舉 k=i..j-1 不會漏解。

依 `l=j-i+1` 遞增填表，子鏈都比當前短；`split[i,j]` 保存最佳 k。狀態數 Theta(n^2)，每格掃 O(n) 個切點，總時間 Theta(n^3)。

更精確地，候選切點總數為 `Σ_(l=2)^n (n−l+1)(l−1)=n(n²−1)/6`。每個候選做常數次表格讀取與算術，這同時給出上、下界，並非只由 O(n^2)×O(n) 推測 Θ。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Matrix-Chain 的填表與括號輸出

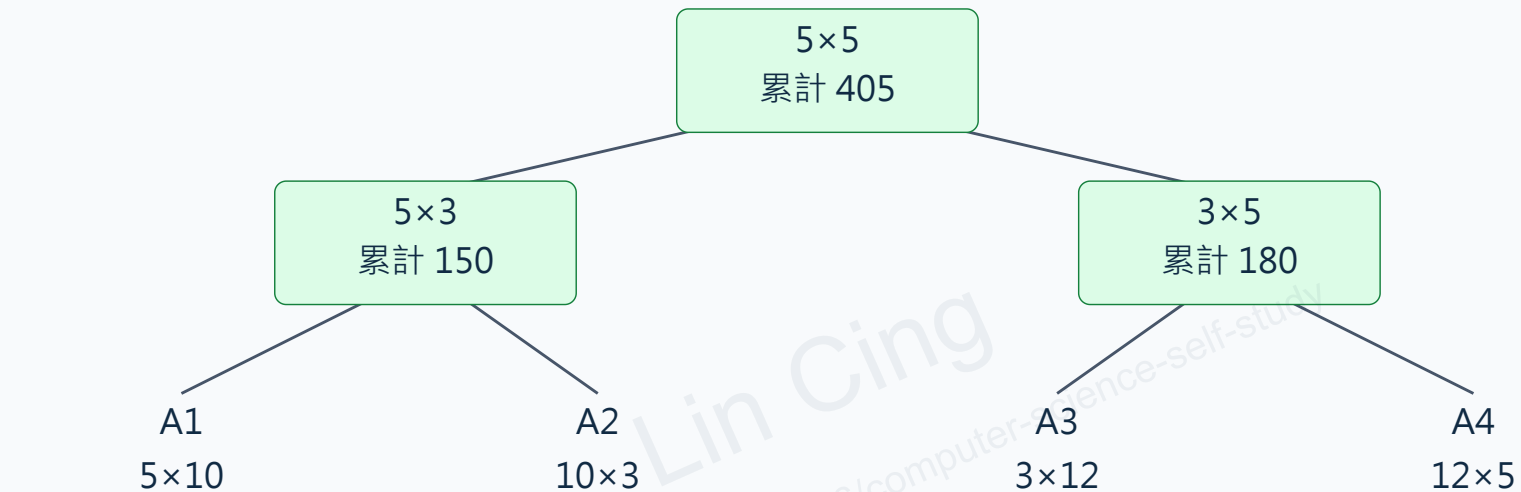
```
for i = 1..n: m[i,i] = 0
for length = 2..n:
    for i = 1..n-length+1:
        j = i+length-1
        m[i,j] = +∞
        for k = i..j-1:
            c = m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
            if c < m[i,j]: m[i,j], split[i,j] = c, k
emit(i,j):
    if i == j: print A_i; return
    k = split[i,j]
    print '('; emit(i,k); emit(k+1,j); print ')'
```

每項轉移皆讀取嚴格較短的區間，所以 length 的順序合法。重建樹有 n 葉、n−1 個內節點，輸出括號需 Θ(n) 次節點操作；不含實際矩陣元素的乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-Chain 的填表與括號輸出 (續)

最優矩陣鏈 ((A1A2)(A3A4))



150 + 180 + (5×3×5) = 405；根的最後乘法只新增 75。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix-Chain 的完整表與括號重建

取 `p=[5,10,3,12,5]`，A1.A4 的 m 上三角如下：

j-i+1	值
1	m11=m22=m33=m44=0
2	m12=150、m23=360、m34=180
3	m13=330、m24=330
4	m14=405

例如 `m24=min(0+180+10*3*5, 360+0+10*12*5)=330`，split[2,4]=2；`m14=min(0+330+5*10*5, 150+180+5*3*5, 330+0+5*12*5)=405`，split[1,4]=2。

回溯 `split[1,4]=2`，得到 `(A1A2)` 與 `(A3A4)`；兩段各以 `split[1,2]=1`、`split[3,4]=3` 展開，最終 `((A1A2)(A3A4))`。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

15.3 Elements of dynamic programming

動態規劃的狀態不是「把大問題換一個名字」。它必須足以描述原問題的剩餘工作，而且每個轉移只依賴更小的狀態。完整證明要依序建立四件事：

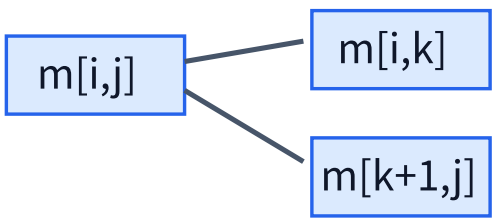
1. 最優解去掉第一個決策後，剩下的是對應子問題的最優解。
2. 每個原問題候選決策都能由已定義的子狀態組合，且沒有漏掉合法解。
3. 基底值正確，且依賴圖無環。
4. 拓撲次序保證填值時所有先決狀態已經算好。

這些條件共同建立轉移正確性與合法計算次序。重疊子問題只說明快取能避免重算，不會單獨保證最佳子結構。若狀態依賴形成 cycle，必須另找有據的求解法，不能直接套一次填表。重建可保存 choice，也可重新檢查完整值表的轉移；只保留最終一個最佳值通常不夠。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

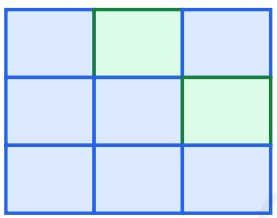
共同骨架：狀態依賴是有向無環圖

1 狀態與先決



先決狀態變小且無環

2 依長度填表



l=1 → 2 → ⋯

只讀較短區間

時間 = 狀態數 × 轉移數

3 保存選擇並回溯

rod：cut[length]

matrix：split[i,j]

LCS：backpointer

BST：root[i,j]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態圖決定快取的成本

rod 的依賴圖有 n+1 個狀態，長度 j 向 0..j−1 各有一個依賴邊，共 n(n+1)/2 條。若每狀態與每條邊只處理常數次，時間是 $\Theta(V+E)=\Theta(n^2)$。

自頂向下 memo 在第一次計算狀態時展開全部候選，之後立即返回已存值。visited 必須獨立於數值，例如以 None 區分未算；不能把合法的負收益當作「未計算」。由小到大填表沒有遞迴堆疊，memo 最多有 n 層呼叫堆疊，兩者另存表均 Θ(n)。

最長簡單路徑的反例不是「任意 cycle 都不能做 DP」。問題在於只存端點無法限制已用頂點。例如兩段各自最長的簡單路徑 u→v、v→w，可能都經過 z，拼接後重複 z 而不再簡單。若加入已訪問集合 S，可作指數狀態 DP；圖是 DAG 時，也能直接按拓撲次序求最長路徑。

Lin Ching https://github.com/xinhi6716/computer-science-self-study

15.4 Longest common subsequence

令 `c[i,j]` 為 `X_i`、`Y_j` 的最長共同子序列長度，空前綴的長度為 0。

若 `x_i=y_j`，任取一個 LCS，最後字元若不是 `x_i`，可把它 append 得到更長共同序列；所以可令最後字元相同，並去掉它得到 `X_{i-1},Y_{j-1}` 的候選。若該前綴存在更長共同序列，加上 x_i 就會超過原 LCS，矛盾，故必是 prefix LCS。

若 `x_i != y_j`，任何共同子序列末字元不可能同時等於這兩個不同字元，因此至少不依賴其中一個末字元。兩種情況合併後：

`c[i,j]=max(c[i-1,j], c[i,j-1])`。

兩個分支已覆蓋全部可能，轉移為 O(1)，完整表為 O(mn)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCS 的兩向界與完整實作

相等時，任一 LCS 若不以共同末字元結尾，就不能使用任一輸入的末位置，故可再接該字元，違反最長。刪掉其末字元得到長度至多 c[i−1,j−1] 的解；反過來，把任一前綴 LCS 接上共同字元也是合法解。兩向界給 c[i,j]=c[i−1,j−1]+1。

```
c[0,0..n] = 0; c[0..m,0] = 0
for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]: c[i,j] = c[i-1,j-1]+1
        else: c[i,j] = max(c[i-1,j],c[i,j-1])
i=m; j=n; result=[]
while i>0 and j>0:
    if X[i]==Y[j]: append X[i] to result; i--; j--
    elif c[i-1,j]>=c[i,j-1]: i--
    else: j--
reverse result
```

保存完整 c 表已足以判斷回溯方向，不一定另存方向表。若只求長度，可沿短字串的維度保留兩列，用 Θ(min(m,n)) 空間；上述回溯則仍依賴完整表，不能直接在兩列上執行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCS 的完整表與回溯

X=ABCBDAB、Y=BDCABA，本例 c[7,6]=4。加上空前綴後的 c 表為：

```
      ε B D C A B A
ε     0 0 0 0 0 0 0
A     0 0 0 0 1 1 1
B     0 1 1 1 1 2 2
C     0 1 1 2 2 2 2
B     0 1 1 2 2 3 3
D     0 1 2 2 2 3 3
A     0 1 2 2 3 3 4
B     0 1 2 2 3 4 4
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCS 的完整表與回溯 （續）

回溯時，表格中的「上」代表 i 減一，「左」代表 j 減一：

當前格	動作	下一格	收集的反序字元
(7,6)	上，兩候選同為 4	(6,6)	空
(6,6)	A=A，左上	(5,5)	A
(5,5)	D≠B，上方值 3 較大	(4,5)	A
(4,5)	B=B，左上	(3,4)	AB
(3,4)	C≠A，左方值 2 較大	(3,3)	AB
(3,3)	C=C，左上	(2,2)	ABC
(2,2)	B≠D，左方值 1 較大	(2,1)	ABC
(2,1)	B=B，左上	(1,0)	ABCB

反向輸出 BCBA。每步至少減一個索引，故回溯 O(m+n)，即使原表有 mn 格也不必掃完。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

LCS 的完整表與回溯 （續）

LCS 的值表與八步回溯

i↓ / j→	0 ε	1 B	2 D	3 C	4 A	5 B	6 A
0 ε	0	0	0	0	0	0	0
1 A	0	0	0	0	1	1	1
2 B	0	1	1	1	1	2	2
3 C	0	1	1	2	2	2	2
4 B	0	1	1	2	2	3	3
5 D	0	1	2	2	2	3	3
6 A	0	1	2	2	3	3	4
7 B	0	1	2	2	3	4	4

綠格為回溯路徑

(7,6) 上
(6,6) 左上，取 A
(5,5) 上
(4,5) 左上，取 B
(3,4) 左
(3,3) 左上，取 C
(2,2) 左
(2,1) 左上，取 B

c[7,6]=4；反序收集 A B C B，反向為 BCBA。

LinCing https://github.com/xinhi6716/computer-science-self-study

15.5 Optimal binary search trees

給定排序鍵 k1<k2<…<kn，須選擇滿足 BST 鍵序的樹形，讓查詢的期望成本最低。成功與失敗查詢都納入目標。

成功鍵 k_i 的機率是 p_i，失敗區間 d_i 的機率是 q_i，總和為 1。搜尋成本以檢查的節點數計算，樹 T 的期望成本是

`sum_i p_i(depth_T(k_i)+1) + sum_i q_i(depth_T(d_i)+1)`。

`e[i,j]` 表 k_i..k_j 的最佳成本，`w[i,j]` 是這段所有鍵與 dummy 的機率和。空段只有一個 dummy：

`e[i,i-1]=w[i,i-1]=q_{i-1}`。

若根選 k_r，左、右子樹各自接到根下，其所有搜尋深度都增加 1，所以成本各增加自己的權重：

`e[i,j]=min_{i<=r<=j}(e[i,r-1]+e[r+1,j]+w[i,j])`。

任一切掉最優子樹再貼入更便宜子樹，都會降低原樹成本，故此 recurrence 完備。

Optimal BST 的填表及成本慣例

```
for i = 1..n+1:
    e[i,i-1] = w[i,i-1] = q[i-1]
for length = 1..n:
    for i = 1..n-length+1:
        j = i+length-1
        w[i,j] = w[i,j-1]+p[j]+q[j]
        e[i,j] = +∞
        for r = i..j:
            c = e[i,r-1]+e[r+1,j]+w[i,j]
            if c < e[i,j]: e[i,j], root[i,j] = c, r
```

本節採 CLRS 的擴充搜尋樹成本，失敗搜尋也計一次 dummy 的檢查。若實作只計真實鍵比較，期望成本為 e[1,n]−Σq_i；這個與樹形無關的常數不影響最優樹。例子的最優成本 2.10，在只計鍵比較模型下為 2.10−0.40=1.70。

重建 `build(i,j)`：若 j<i，輸出區間 dummy d_(i−1)；否則 r=root[i,j]，輸出鍵 k_r 及 build(i,r−1)、build(r+1,j)。空區間必須保留，才能正確交付失敗搜尋的葉。

Optimal BST 的完整表與樹重建

取三鍵 p=(0.20,0.30,0.10)，dummy q=(0.10,0.05,0.15,0.10)。先算：

區間	w	e	最優根
[1,1]	0.35	0.50	k1
[2,2]	0.50	0.70	k2
[3,3]	0.35	0.60	k3
[1,2]	0.80	1.45	k2
[2,3]	0.70	1.35	k2
[1,3]	1.00	2.10	k2

例如 `e[1,2]` 以 k2 為根時，成本 `e[1,1]+e[3,2]+w[1,2]=0.50+0.15+0.80=1.45`。root[1,3]=2；左子問題 root[1,1]=1，右 root[3,3]=3，故樹根 k2，左右 child 分別 k1、k3，dummy 依 BST 區間掛放。

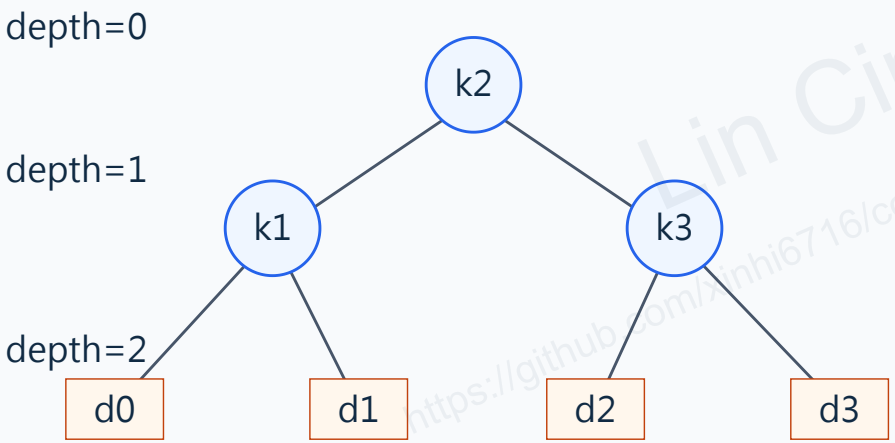
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimal BST 的完整表與樹重建 （續）

以長度填表，e、w、root 各 Theta(n^2) 格；每格枚舉最多 n 個根，共 Theta(n^3)。本章偽碼採這個已證明的版本。

候選根總數為 Σ_(l=1)^n (n−l+1)l=n(n+1)(n+2)/6。w 每格以 w[i,j−1]+p_j+q_j 常數時間更新，若在每個候選根內重掃整個機率區間，會多乘一個 n，不能沿用本分析。

最佳 BST 與四個失敗搜尋區間



期望成本 = 機率 × (深度+1)

k2：0.30 × 1 = 0.30

k1,k3：(0.20+0.10) × 2

d0..d3：0.40 × 3 = 1.20

總計 0.30+0.60+1.20 = 2.10

d0: <k1 d1: (k1,k2) d2: (k2,k3) d3: >k3

Lin Cing https://github.com/xinhi6716/computer-science-self-study

序列比對：LCS 的陌生變形

全域序列比對允許 match、mismatch 與 gap，最後一欄可能是：

- `(x_i,y_j)` 配對，得分 match 或 mismatch；
- `x_i` 配 gap；
- `y_j` 配 gap。

若 `F[i,j]` 是前綴最高分，三種情形給出

`F[i,j]=max(F[i-1,j-1]+score(x_i,y_j), F[i-1,j]+gap, F[i,j-1]+gap)`。

它與 LCS 共用前綴狀態與回溯骨架，但不再只有相等/不等兩分支。對 X=ACGTA、Y=AGCAC，match=3、mismatch=-2、gap=-3，最佳分數為 1，對齊是 `ACGTA- / A-GCAC`。每一個 gap 都消耗一個固定分數，不能只比較長度相同的字元。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無界背包：容量是狀態的一部分

若物品可重複取，且重量 s_i、價值 v_i，令 f[c] 為重量恰為 c 的最大價值：

`f[0]=0`，`f[c]=max_{i:s_i<=c}(f[c-s_i]+v_i)`。

最後要對 `0<=c<=C` 取最大值；若規格要求恰好裝滿，只取 f[C]。若轉移改成 `f[c]=max(f[c-1],...)`，則表示容量至多 c，需明確區分語意。

以 A=(3,5)、B=(4,7)、C=(7,12)、C=13 為例，

`f[0..13]=[0,-∞,-∞,5,7,-∞,10,12,14,15,17,19,21,22]`。

最優在容量 13，選 A+A+A+B，價值 22。按 value/weight 最大的 greedy 會取 B×3，共 21，故一般比值 greedy 不正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 15。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§15.1-15.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study